01. how to find the verson of pandas

In [67]:
```
import pandas as pd
pd.__version__
```

Out[67]:
```
'2.3.2'
```

In [68]:
```
pd.show_versions()
```

```
INSTALLED VERSIONS
------------------
commit                : 4665c10899bc413b639194f6fb8665a5c70f7db5
python                : 3.13.7
python-bits           : 64
OS                    : Windows
OS-release            : 11
Version               : 10.0.26200
machine               : AMD64
processor             : Intel64 Family 6 Model 140 Stepping 1, GenuineIntel
byteorder             : little
LC_ALL                : None
LANG                  : None
LOCALE                : English_United States.1252

pandas                : 2.3.2
numpy                 : 2.3.2
pytz                  : 2025.2
dateutil              : 2.9.0.post0
pip                   : 26.2.1
Cython                : None
sphinx                : None
IPython               : 9.5.0
adbc-driver-postgresql: None
adbc-driver-sqlite    : None
bs4                   : 4.13.5
blosc                 : None
bottleneck            : None
dataframe-api-compat  : None
fastparquet           : None
fsspec                : None
html5lib              : None
hypothesis            : None
gcsfs                 : None
jinja2                : 3.1.6
lxml.etree            : None
matplotlib            : 3.10.6
numba                 : None
numexpr               : None
odfpy                 : None
openpyxl              : 3.1.5
pandas_gbq            : None
psycopg2              : None
pymysql               : None
pyarrow               : 21.0.0
pyreadstat            : None
pytest                : None
python-calamine       : None
pyxlsb                : None
s3fs                  : None
scipy                 : 1.16.1
sqlalchemy            : None
tables                : None
tabulate              : None
xarray                : None
xlrd                  : 2.0.2
```

```
xlsxwriter            : 3.2.5
zstandard             : None
tzdata                : 2025.2
qtpy                  : None
pyqt5                 : None
```

02. Make a dataframe

In [69]:
```
df = pd.DataFrame({'A col': [1,2,3,4,5,6], 'B col':[7,8,9,10,11,12]})
df
```

Out[69]:

	A col	B col
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

In [70]:
```
# numpy array use to create dataframe
import numpy as np
```

In [71]:
```
arr = np.array([[1,2,3],[4,5,6],[7,8,9]])
pd.DataFrame(arr)
```

Out[71]:

	0	1	2
0	1	2	3
1	4	5	6
2	7	8	9

In [72]:
```
pd.DataFrame(np.random.rand(4,8))
```

Out[72]:

	0	1	2	3	4	5	6	7
0	0.909172	0.193290	0.230401	0.746265	0.552617	0.583336	0.876900	0.440555
1	0.447745	0.702243	0.327393	0.107964	0.948954	0.083707	0.242962	0.355433
2	0.352579	0.752943	0.059449	0.951612	0.949069	0.344027	0.402003	0.692030
3	0.296160	0.936689	0.552735	0.032967	0.571214	0.696929	0.190978	0.296935

In [73]:
```
pd.DataFrame(np.random.rand(4,8), columns=list('ABCDEFGH'))
```

Out[73]:

	A	B	C	D	E	F	G	H
0	0.471774	0.617953	0.507436	0.439364	0.148006	0.482799	0.265080	0.166045
1	0.644366	0.292537	0.601648	0.404826	0.081122	0.401185	0.095883	0.398915
2	0.211554	0.514719	0.252685	0.203112	0.500059	0.587604	0.916397	0.592763
3	0.784466	0.673210	0.698749	0.603807	0.578109	0.000535	0.357686	0.978694

In [74]:
```
pd.DataFrame(np.random.rand(5,80))
```

Out[74]:

	0	1	2	3	4	5	6	7	8
0	0.800213	0.256767	0.689561	0.520890	0.017350	0.702011	0.910936	0.443966	0.534609
1	0.954006	0.290715	0.004533	0.935298	0.136330	0.304445	0.068710	0.548665	0.555497
2	0.516288	0.616490	0.494350	0.642681	0.206254	0.292537	0.545939	0.424979	0.626085
3	0.785843	0.159218	0.594958	0.400058	0.062742	0.268516	0.345182	0.919103	0.359692
4	0.389085	0.300706	0.960296	0.575376	0.532292	0.861725	0.050259	0.496250	0.957049

5 rows × 80 columns

3. How to rename columns?

In [75]:
```
df = pd.DataFrame({'A col': [1,2,3,4,5,6], 'B col':[7,8,9,10,11,12]})
df.head()
```

Out[75]:

	A col	B col
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11

In [76]:
```
df.rename(columns={'A col': 'col_a', 'B col': 'col_b'}, inplace=True)
df
```

Out[76]:

	col_a	col_b
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

In [77]:
```
df.columns = ['col_aa','col_bb']
df
```

Out[77]:

	col_aa	col_bb
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

In [78]:
```
# to replace character or string
df.columns = df.columns.str.replace('_',' ')
df
```

Out[78]:

	col aa	col bb
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

In [79]:
```
# to replace character or string
df.columns= df.columns.str.replace(' ','_')
df
```

Out[79]:

	col_aa	col_bb
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

In [80]:
```
df = df.add_prefix('ad_')
df
```

Out[80]:

	ad_col_aa	ad_col_bb
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

In [81]:
```
df = df.add_suffix('ad_')
df
```

Out[81]:

	ad_col_aaad_	ad_col_bbad_
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

In [82]:
```
df.columns = ['col_a', 'col_b']
df
```

Out[82]:

	col_a	col_b
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

4. using template data

In [ ]:

In [83]:
```
import pandas as pd
import numpy as np
import seaborn as sns
```

In [87]:
```
kashti = sns.load_dataset('titanic')
kashti.head()
```

Out[87]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

In [88]:
```
phool = sns.load_dataset('iris')
phool.head()
```

Out[88]:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

In [90]:
```
df = sns.load_dataset('tips')
df.head()
```

Out[90]:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

In [91]:
```
df.describe()
```

Out[91]:

	total_bill	tip	size
count	244.000000	244.000000	244.000000
mean	19.785943	2.998279	2.569672
std	8.902412	1.383638	0.951100
min	3.070000	1.000000	1.000000
25%	13.347500	2.000000	2.000000
50%	17.795000	2.900000	2.000000
75%	24.127500	3.562500	3.000000
max	50.810000	10.000000	6.000000

In [92]:
```
# saving template dataset
df.to_csv('tips_save.csv')
df.to_excel('tips_save.xlsx')
```

5. importing own data

In [97]:
```
import pandas as pd
df = pd.read_csv('tips_save.csv')
df.head()
```

Out[97]:

	Unnamed: 0	total_bill	tip	sex	smoker	day	time	size
0	0	16.99	1.01	Female	No	Sun	Dinner	2
1	1	10.34	1.66	Male	No	Sun	Dinner	3
2	2	21.01	3.50	Male	No	Sun	Dinner	3
3	3	23.68	3.31	Male	No	Sun	Dinner	2
4	4	24.59	3.61	Female	No	Sun	Dinner	4

In [99]:
```
import pandas as pd
df = pd.read_excel('tips_save.xlsx')
df.head()
```

Out[99]:

	Unnamed: 0	total_bill	tip	sex	smoker	day	time	size
0	0	16.99	1.01	Female	No	Sun	Dinner	2
1	1	10.34	1.66	Male	No	Sun	Dinner	3
2	2	21.01	3.50	Male	No	Sun	Dinner	3
3	3	23.68	3.31	Male	No	Sun	Dinner	2
4	4	24.59	3.61	Female	No	Sun	Dinner	4

6. Reverse row orders

In [100…
```
import seaborn as sns
import pandas as pd
df = sns.load_dataset('titanic')
df.head()
```

Out[100…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	F
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	F
3	1	1	female	35.0	1	0	53.1000	S	First	woman	F
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

In [107…
```
df.loc[::-1].head()
```

Out[107…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult
890	0	3	male	32.0	0	0	7.75	Q	Third	man	
889	1	1	male	26.0	0	0	30.00	C	First	man	
888	0	3	female	NaN	1	2	23.45	S	Third	woman	
887	1	1	female	19.0	0	0	30.00	S	First	woman	
886	0	2	male	27.0	0	0	13.00	S	Second	man	

In [103…

```
df.loc[::-6].head()
```

Out[103…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	ad
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	
884	0	3	male	25.0	0	0	7.0500	S	Third	man	
878	0	3	male	NaN	0	0	7.8958	S	Third	man	
872	0	1	male	33.0	0	0	5.0000	S	First	man	
866	1	2	female	27.0	1	0	13.8583	C	Second	woman	

In [108…

```
df.loc[::-1].reset_index(drop=True).head()
```

Out[108…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	32.0	0	0	7.75	Q	Third	man	T
1	1	1	male	26.0	0	0	30.00	C	First	man	T
2	0	3	female	NaN	1	2	23.45	S	Third	woman	F
3	1	1	female	19.0	0	0	30.00	S	First	woman	F
4	0	2	male	27.0	0	0	13.00	S	Second	man	T

7. Reverse column order

In [109…

```
df.loc[:, ::-1].head()
```

Out[109…

	alone	alive	embark_town	deck	adult_male	who	class	embarked	fare	parch
0	False	no	Southampton	NaN	True	man	Third	S	7.2500	0
1	False	yes	Cherbourg	C	False	woman	First	C	71.2833	0
2	True	yes	Southampton	NaN	False	woman	Third	S	7.9250	0
3	False	yes	Southampton	C	False	woman	First	S	53.1000	0
4	True	no	Southampton	NaN	True	man	Third	S	8.0500	0

8. select a column by data type

In [111…

```
df.dtypes
```

Out[111…

```
survived          int64
pclass            int64
sex              object
age             float64
sibsp             int64
parch             int64
fare            float64
embarked         object
class          category
who              object
adult_male         bool
deck           category
embark_town      object
alive            object
alone              bool
dtype: object
```

In [ ]:

```
#columns with numerics
df.select_dtypes(include=['number']).head()
```

Out[ ]:

	survived	pclass	age	sibsp	parch	fare
0	0	3	22.0	1	0	7.2500
1	1	1	38.0	1	0	71.2833
2	1	3	26.0	0	0	7.9250
3	1	1	35.0	1	0	53.1000
4	0	3	35.0	0	0	8.0500

In [ ]:

```
# columns with object type
df.select_dtypes(include=['object']).head()
```

Out[ ]:

	sex	embarked	who	embark_town	alive
0	male	S	man	Southampton	no
1	female	C	woman	Cherbourg	yes
2	female	S	woman	Southampton	yes
3	female	S	woman	Southampton	yes
4	male	S	man	Southampton	no

In [114…
```
# cboth object and number
df.select_dtypes(include=['number','object']).head()
```

Out[114…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	who	embark_town
0	0	3	male	22.0	1	0	7.2500	S	man	Southampton
1	1	1	female	38.0	1	0	71.2833	C	woman	Cherbourg
2	1	3	female	26.0	0	0	7.9250	S	woman	Southampton
3	1	1	female	35.0	1	0	53.1000	S	woman	Southampton
4	0	3	male	35.0	0	0	8.0500	S	man	Southampton

In [115…
```
df.select_dtypes(exclude=['number']).head()
```

Out[115…

	sex	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	male	S	Third	man	True	NaN	Southampton	no	False
1	female	C	First	woman	False	C	Cherbourg	yes	False
2	female	S	Third	woman	False	NaN	Southampton	yes	True
3	female	S	First	woman	False	C	Southampton	yes	False
4	male	S	Third	man	True	NaN	Southampton	no	True

In [116…
```
df.select_dtypes(exclude=['object']).head()
```

Out[116…

	survived	pclass	age	sibsp	parch	fare	class	adult_male	deck	alone
0	0	3	22.0	1	0	7.2500	Third	True	NaN	False
1	1	1	38.0	1	0	71.2833	First	False	C	False
2	1	3	26.0	0	0	7.9250	Third	False	NaN	True
3	1	1	35.0	1	0	53.1000	First	False	C	False
4	0	3	35.0	0	0	8.0500	Third	True	NaN	True

9. Convert string to numbers

In [121…
```
df = pd.DataFrame({'col_a':['1','2','3','4','5','6'],
                   'col_b':['7','8','9','10','11','12']})
df
```

Out[121…

	col_a	col_b
0	1	7
1	2	8
2	3	9
3	4	10
4	5	11
5	6	12

In [122…
```
df.dtypes
```

Out[122…
```
col_a    object
col_b    object
dtype: object
```

In [125…
```
df.astype({'col_a':'int64', 'col_b':'int64'}).dtypes
```

Out[125…
```
col_a    int64
col_b    int64
dtype: object
```

In [129…
```
pd.to_numeric(df['col_a'],errors='coerce')
df.dtypes
```

Out[129…
```
col_a    object
col_b    object
dtype: object
```

10. Reduce Dataframe type

In [130…
```
df=sns.load_dataset('titanic')
df.shape
```

Out[130…
```
(891, 15)
```

In [131…
```
df
```

Out[131…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	ad
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	
…	…	…	…	…	…	…	…	…	…	…	
886	0	2	male	27.0	0	0	13.0000	S	Second	man	
887	1	1	female	19.0	0	0	30.0000	S	First	woman	
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	
889	1	1	male	26.0	0	0	30.0000	C	First	man	
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	

891 rows × 15 columns

In [134…

```
#take some data
df.sample(frac=0.1).shape
df.info
```

Out[134…

```
<bound method DataFrame.info of      survived  pclass     sex   age  sibsp  parch
fare embarked   class  \
0           0       3    male  22.0      1      0   7.2500        S   Third
1           1       1  female  38.0      1      0  71.2833        C   First
2           1       3  female  26.0      0      0   7.9250        S   Third
3           1       1  female  35.0      1      0  53.1000        S   First
4           0       3    male  35.0      0      0   8.0500        S   Third
..        ...     ...     ...   ...    ...    ...      ...      ...     ...
886         0       2    male  27.0      0      0  13.0000        S  Second
887         1       1  female  19.0      0      0  30.0000        S   First
888         0       3  female   NaN      1      2  23.4500        S   Third
889         1       1    male  26.0      0      0  30.0000        C   First
890         0       3    male  32.0      0      0   7.7500        Q   Third

       who  adult_male deck  embark_town alive  alone
0      man        True  NaN  Southampton    no  False
1    woman       False    C    Cherbourg   yes  False
2    woman       False  NaN  Southampton   yes   True
3    woman       False    C  Southampton   yes  False
4      man        True  NaN  Southampton    no   True
..     ...         ...  ...          ...   ...    ...
886    man        True  NaN  Southampton    no   True
887  woman       False    B  Southampton   yes   True
888  woman       False  NaN  Southampton    no  False
889    man        True    C    Cherbourg   yes   True
890    man        True  NaN   Queenstown    no   True

[891 rows x 15 columns]>
```

11. Copy data from clip board

In [145…

```
# downloiad dataset
import seaborn as sns
import pandas as pd
df = sns.load_dataset('Titanic')
df.to_excel('kashit.xlsx')
```

In [ ]:

```
df = pd.read_clipboard()
df
```

Out[ ]:

	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive
0	1	0	7.25	S	Third	man	True	NaN	Southampton	no
1	1	0	71.28	C	First	woman	False	C	Cherbourg	yes
2	0	0	7.93	S	Third	woman	False	NaN	Southampton	yes
3	1	0	53.10	S	First	woman	False	C	Southampton	yes
4	0	0	8.05	S	Third	man	True	NaN	Southampton	no
5	0	0	8.46	Q	Third	man	True	NaN	Queenstown	no
6	0	0	51.86	S	First	man	True	E	Southampton	no
7	3	1	21.08	S	Third	child	False	NaN	Southampton	no
8	0	2	11.13	S	Third	woman	False	NaN	Southampton	yes
9	1	0	30.07	C	Second	child	False	NaN	Cherbourg	yes
10	1	1	16.70	S	Third	child	False	G	Southampton	yes
11	0	0	26.55	S	First	woman	False	C	Southampton	yes
12	0	0	8.05	S	Third	man	True	NaN	Southampton	no
13	1	5	31.28	S	Third	man	True	NaN	Southampton	no
14	0	0	7.85	S	Third	child	False	NaN	Southampton	no
15	0	0	16.00	S	Second	woman	False	NaN	Southampton	yes
16	4	1	29.13	Q	Third	child	False	NaN	Queenstown	no
17	0	0	13.00	S	Second	man	True	NaN	Southampton	yes

In [139…

```
df = pd.read_clipboard('selected.xlsx')
df
```

Out[139...

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult
0	0	3	male	22.0	1	0	7.25	S	Third	man	
1	1	1	female	38.0	1	0	71.28	C	First	woman	
2	1	3	female	26.0	0	0	7.93	S	Third	woman	
3	1	1	female	35.0	1	0	53.10	S	First	woman	
4	0	3	male	35.0	0	0	8.05	S	Third	man	
5	0	3	male	NaN	0	0	8.46	Q	Third	man	
6	0	1	male	54.0	0	0	51.86	S	First	man	
7	0	3	male	2.0	3	1	21.08	S	Third	child	
8	1	3	female	27.0	0	2	11.13	S	Third	woman	
9	1	2	female	14.0	1	0	30.07	C	Second	child	
10	1	3	female	4.0	1	1	16.70	S	Third	child	
11	1	1	female	58.0	0	0	26.55	S	First	woman	
12	0	3	male	20.0	0	0	8.05	S	Third	man	
13	0	3	male	39.0	1	5	31.28	S	Third	man	
14	0	3	female	14.0	0	0	7.85	S	Third	child	
15	1	2	female	55.0	0	0	16.00	S	Second	woman	
16	0	3	male	2.0	4	1	29.13	Q	Third	child	
17	1	2	male	NaN	0	0	13.00	S	Second	man	
18	0	3	female	31.0	1	0	18.00	S	Third	woman	
19	1	3	female	NaN	0	0	7.23	C	Third	woman	
20	0	2	male	35.0	0	0	26.00	S	Second	man	
21	1	2	male	34.0	0	0	13.00	S	Second	man	
22	1	3	female	15.0	0	0	8.03	Q	Third	child	
23	1	1	male	28.0	0	0	35.50	S	First	man	
24	0	3	female	8.0	3	1	21.08	S	Third	child	
25	1	3	female	38.0	1	5	31.39	S	Third	woman	
26	0	3	male	NaN	0	0	7.23	C	Third	man	
27	0	1	male	19.0	3	2	263.00	S	First	man	
28	1	3	female	NaN	0	0	7.88	Q	Third	woman	
29	0	3	male	NaN	0	0	7.90	S	Third	man	

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult
30	0	1	male	40.0	0	0	27.72	C	First	man	
31	1	1	female	NaN	1	0	146.52	C	First	woman	
32	1	3	female	NaN	0	0	7.75	Q	Third	woman	
33	0	2	male	66.0	0	0	10.50	S	Second	man	
34	0	1	male	28.0	1	0	82.17	C	First	man	

In [146…
```
df.shape
```

Out[146…
```
(891, 15)
```

12. Split datframe into two

In [157…
```
kashti_1 = sns.load_dataset('Titanic')
kashti_1 = df.sample(frac=0.5, random_state=1)
kashti_1.shape
```

Out[157…
```
(446, 15)
```

In [158…
```
kashti_2 = sns.load_dataset('Titanic')
kashti_2 = df.drop(kashti_1.index)
kashti_2.shape
```

Out[158…
```
(445, 15)
```

In [153…
```
kashti_1.head()
```

Out[153…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adu
862	1	1	female	48.0	0	0	25.9292	S	First	woman	
223	0	3	male	NaN	0	0	7.8958	S	Third	man	
84	1	2	female	17.0	0	0	10.5000	S	Second	woman	
680	0	3	female	NaN	0	0	8.1375	Q	Third	woman	
535	1	2	female	7.0	0	2	26.2500	S	Second	child	

In [154…
```
kashti_2.head()
```

Out[154…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adul
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
7	0	3	male	2.0	3	1	21.0750	S	Third	child	
10	1	3	female	4.0	1	1	16.7000	S	Third	child	
15	1	2	female	55.0	0	0	16.0000	S	Second	woman	
18	0	3	female	31.0	1	0	18.0000	S	Third	woman	

In [159…
```
len(kashti_1) + len(kashti_2)
```

Out[159… 891

13. Join two datasets

In [165…
```
import pandas as pd

df1 = pd.concat([kashti_1, kashti_2],ignore_index=True)

print(df1.shape)
```

```
(891, 15)
```

14. filtring a dataset

In [182…
```
sns.load_dataset('titanic')
df.head()
```

Out[182…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

In [183…
```
df.sex.unique
```

Out[183…
```
<bound method Series.unique of 0        male
1      female
2      female
3      female
4        male
       ...
886      male
887    female
888    female
889      male
890      male
Name: sex, Length: 891, dtype: object>
```

In [185…
```
df.age.unique()
```

Out[185…
```
array([22.  , 38.  , 26.  , 35.  ,   nan, 54.  ,  2.  , 27.  , 14.  ,
        4.  , 58.  , 20.  , 39.  , 55.  , 31.  , 34.  , 15.  , 28.  ,
        8.  , 19.  , 40.  , 66.  , 42.  , 21.  , 18.  ,  3.  ,  7.  ,
       49.  , 29.  , 65.  , 28.5 ,  5.  , 11.  , 45.  , 17.  , 32.  ,
       16.  , 25.  ,  0.83, 30.  , 33.  , 23.  , 24.  , 46.  , 59.  ,
       71.  , 37.  , 47.  , 14.5 , 70.5 , 32.5 , 12.  ,  9.  , 36.5 ,
       51.  , 55.5 , 40.5 , 44.  ,  1.  , 61.  , 56.  , 50.  , 36.  ,
       45.5 , 20.5 , 62.  , 41.  , 52.  , 63.  , 23.5 ,  0.92, 43.  ,
       60.  , 10.  , 64.  , 13.  , 48.  ,  0.75, 53.  , 57.  , 80.  ,
       70.  , 24.5 ,  6.  ,  0.67, 30.5 ,  0.42, 34.5 , 74.  ])
```

In [186…
```
df.embark_town.unique()
```

Out[186…
```
array(['Southampton', 'Cherbourg', 'Queenstown', nan], dtype=object)
```

In [187…
```
df [(df.sex=='female')].head()
```

Out[187…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
8	1	3	female	27.0	0	2	11.1333	S	Third	woman	
9	1	2	female	14.0	1	0	30.0708	C	Second	child	

In [188…
```
df.age.unique()
```

Out[188…
```
array([22.  , 38.  , 26.  , 35.  ,   nan, 54.  ,  2.  , 27.  , 14.  ,
        4.  , 58.  , 20.  , 39.  , 55.  , 31.  , 34.  , 15.  , 28.  ,
        8.  , 19.  , 40.  , 66.  , 42.  , 21.  , 18.  ,  3.  ,  7.  ,
       49.  , 29.  , 65.  , 28.5 ,  5.  , 11.  , 45.  , 17.  , 32.  ,
       16.  , 25.  ,  0.83, 30.  , 33.  , 23.  , 24.  , 46.  , 59.  ,
       71.  , 37.  , 47.  , 14.5 , 70.5 , 32.5 , 12.  ,  9.  , 36.5 ,
       51.  , 55.5 , 40.5 , 44.  ,  1.  , 61.  , 56.  , 50.  , 36.  ,
       45.5 , 20.5 , 62.  , 41.  , 52.  , 63.  , 23.5 ,  0.92, 43.  ,
       60.  , 10.  , 64.  , 13.  , 48.  ,  0.75, 53.  , 57.  , 80.  ,
       70.  , 24.5 ,  6.  ,  0.67, 30.5 ,  0.42, 34.5 , 74.  ])
```

In [189…
```
df.embarked.unique()
```

Out[189…
```
array(['S', 'C', 'Q', nan], dtype=object)
```

In [194…
```
df.embark_town.unique()
df[(df.embark_town=='Southampton')].shape
```

Out[194…
```
(644, 15)
```

In [197…
```
df[(df.embark_town=='Southampton') &
   (df.sex=='female')
   ].head()
```

Out[197…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
8	1	3	female	27.0	0	2	11.1333	S	Third	woman	
10	1	3	female	4.0	1	1	16.7000	S	Third	child	
11	1	1	female	58.0	0	0	26.5500	S	First	woman	

In [199…
```
df[((df.embark_town=='Southampton') |
   (df.embark_town=='Queenstown')) &
   (df.sex=='female')
   ]
```

Out[199…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	ad
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
8	1	3	female	27.0	0	2	11.1333	S	Third	woman	
10	1	3	female	4.0	1	1	16.7000	S	Third	child	
11	1	1	female	58.0	0	0	26.5500	S	First	woman	
…	…	…	…	…	…	…	…	…	…	…	
880	1	2	female	25.0	0	1	26.0000	S	Second	woman	
882	0	3	female	22.0	0	0	10.5167	S	Third	woman	
885	0	3	female	39.0	0	5	29.1250	Q	Third	woman	
887	1	1	female	19.0	0	0	30.0000	S	First	woman	
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	

239 rows × 15 columns

In [204…

```
df[df.embark_town.isin(['Queenstown','Southhampton'])].head
```

Out[204…

```
<bound method NDFrame.head of      survived  pclass     sex   age  sibsp  parch
fare embarked  class  \
5           0       3    male   NaN      0      0   8.4583        Q  Third
16          0       3    male   2.0      4      1  29.1250        Q  Third
22          1       3  female  15.0      0      0   8.0292        Q  Third
28          1       3  female   NaN      0      0   7.8792        Q  Third
32          1       3  female   NaN      0      0   7.7500        Q  Third
..        ...     ...     ...   ...    ...    ...      ...      ...    ...
790         0       3    male   NaN      0      0   7.7500        Q  Third
825         0       3    male   NaN      0      0   6.9500        Q  Third
828         1       3    male   NaN      0      0   7.7500        Q  Third
885         0       3  female  39.0      0      5  29.1250        Q  Third
890         0       3    male  32.0      0      0   7.7500        Q  Third

       who  adult_male deck embark_town alive  alone
5      man        True  NaN  Queenstown    no   True
16   child       False  NaN  Queenstown    no  False
22   child       False  NaN  Queenstown   yes   True
28   woman       False  NaN  Queenstown   yes   True
32   woman       False  NaN  Queenstown   yes   True
..     ...         ...  ...         ...   ...    ...
790    man        True  NaN  Queenstown    no   True
825    man        True  NaN  Queenstown    no   True
828    man        True  NaN  Queenstown   yes   True
885  woman       False  NaN  Queenstown    no  False
890    man        True  NaN  Queenstown    no   True

[77 rows x 15 columns]>
```

In [205…
```
df[df.age > 30].shape
```

Out[205…
```
(305, 15)
```

In [206…
```
df[df.age > 30]
```

Out[206…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	
6	0	1	male	54.0	0	0	51.8625	S	First	man	
11	1	1	female	58.0	0	0	26.5500	S	First	woman	
…	…	…	…	…	…	…	…	…	…	…	
873	0	3	male	47.0	0	0	9.0000	S	Third	man	
879	1	1	female	56.0	0	1	83.1583	C	First	woman	
881	0	3	male	33.0	0	0	7.8958	S	Third	man	
885	0	3	female	39.0	0	5	29.1250	Q	Third	woman	
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	

305 rows × 15 columns

15. Filtring by Larhe categories

In [207…
```
df.embark_town.value_counts()
```

Out[207…
```
embark_town
Southampton    644
Cherbourg      168
Queenstown      77
Name: count, dtype: int64
```

In [208…
```
df.age.value_counts()
```

Out[208…
```
age
24.00    30
22.00    27
18.00    26
28.00    25
30.00    25
         ..
24.50     1
0.67      1
0.42      1
34.50     1
74.00     1
Name: count, Length: 88, dtype: int64
```

In [211…
```
df.sex.value_counts().nlargest()
```

Out[211…
```
sex
male      577
female    314
Name: count, dtype: int64
```

In [213…
```
df.age.value_counts().nlargest(3)
```

Out[213…
```
age
24.0    30
22.0    27
18.0    26
Name: count, dtype: int64
```

In [215…
```
df.age.value_counts().nlargest(3).index
```

Out[215…
```
Index([24.0, 22.0, 18.0], dtype='float64', name='age')
```

In [219…
```
df.who.value_counts().nlargest(3).index
```

Out[219…
```
Index(['man', 'woman', 'child'], dtype='object', name='who')
```

In [222…
```
counts = df.embark_town.value_counts()
counts.nlargest
```

Out[222…
```
<bound method Series.nlargest of embark_town
Southampton    644
Cherbourg      168
Queenstown      77
Name: count, dtype: int64>
```

In [224…
```
counts = df.who.value_counts()
counts.nlargest
```

Out[224…
```
<bound method Series.nlargest of who
man      537
woman    271
child     83
Name: count, dtype: int64>
```

In [226…
```
df[df.who.isin(counts.nlargest(3).index)].head()
```

Out[226…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	F
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	F
3	1	1	female	35.0	1	0	53.1000	S	First	woman	F
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

In [227…
```
df[df.who.isin(counts.nlargest(1).index)].head()
```

Out[227…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True
5	0	3	male	NaN	0	0	8.4583	Q	Third	man	True
6	0	1	male	54.0	0	0	51.8625	S	First	man	True
12	0	3	male	20.0	0	0	8.0500	S	Third	man	True

16. Spliting a string into multiple columns

In [258…
```
import pandas as pd
df=pd.DataFrame({'name':['Muhammmad Khan', 'Ahmad Hassan', 'Sajjad Rana', 'Ali Khan
                                    'Location':['Lahore, Pakistan','Multan, Pakista
df
```

Out[258…

	name	Location
0	Muhammmad Khan	Lahore, Pakistan
1	Ahmad Hassan	Multan, Pakistan
2	Sajjad Rana	Karachi, Pakistan
3	Ali Khan	Faisalabad, Pakistan

In [259…
```
# spliting a column into two columns
# adding new columns
df[["first_name", "last_name"]] = df.name.str.split(' ', expand=True)
df
```

Out[259…

	name	Location	first_name	last_name
0	Muhammmad Khan	Lahore, Pakistan	Muhammmad	Khan
1	Ahmad Hassan	Multan, Pakistan	Ahmad	Hassan
2	Sajjad Rana	Karachi, Pakistan	Sajjad	Rana
3	Ali Khan	Faisalabad, Pakistan	Ali	Khan

In [260…

```
df[["City", "Country"]] = df.Location.str.split(', ', expand=True)
df
```

Out[260…

	name	Location	first_name	last_name	City	Country
0	Muhammmad Khan	Lahore, Pakistan	Muhammmad	Khan	Lahore	Pakistan
1	Ahmad Hassan	Multan, Pakistan	Ahmad	Hassan	Multan	Pakistan
2	Sajjad Rana	Karachi, Pakistan	Sajjad	Rana	Karachi	Pakistan
3	Ali Khan	Faisalabad, Pakistan	Ali	Khan	Faisalabad	Pakistan

In [261…

```
#refine data manipulation
df = df[['first_name', 'last_name','City','Country']]
df
```

Out[261…

	first_name	last_name	City	Country
0	Muhammmad	Khan	Lahore	Pakistan
1	Ahmad	Hassan	Multan	Pakistan
2	Sajjad	Rana	Karachi	Pakistan
3	Ali	Khan	Faisalabad	Pakistan

17. Aggrigate by multiple groups or functions

In [286…

```
# libraries
import pandas as pd
import seaborn as sns

# import data set
df = sns.load_dataset('titanic')
df
```

Out[286...

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	ad
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	
...	...	...	...	...	...	...	...	...	...	...	
886	0	2	male	27.0	0	0	13.0000	S	Second	man	
887	1	1	female	19.0	0	0	30.0000	S	First	woman	
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	
889	1	1	male	26.0	0	0	30.0000	C	First	man	
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	

891 rows × 15 columns

In [287...

```
df.groupby('who').count()
```

Out[287...

who	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	adult_male	deck
child	83	83	83	83	83	83	83	83	83	83	13
man	537	537	537	413	537	537	537	537	537	537	99
woman	271	271	271	218	271	271	271	269	271	271	91

In [288...

```
df.groupby('sex').count()
```

Out[288...

sex	survived	pclass	age	sibsp	parch	fare	embarked	class	who	adult_male	decl
female	314	314	261	314	314	314	312	314	314	314	97
male	577	577	453	577	577	577	577	577	577	577	106

In [289...

```
df.groupby('sex').count()
```

Out[289…

sex	survived	pclass	age	sibsp	parch	fare	embarked	class	who	adult_male	decl
female	314	314	261	314	314	314	312	314	314	314	97
male	577	577	453	577	577	577	577	577	577	577	106

In [290…
```
len(df.sex)
```

Out[290… 891

In [291…
```
len(df.groupby('sex'))
```

Out[291… 2

In [292…
```
len(df.groupby('pclass'))
```

Out[292… 3

In [293…
```
df.head()
```

Out[293…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

In [294…
```
df.groupby(['sex','pclass','who']).count()
```

Out[294...

sex	pclass	who	survived	age	sibsp	parch	fare	embarked	class	adult_male	d
female	1	child	3	3	3	3	3	3	3	3	
		woman	91	82	91	91	91	89	91	91	
	2	child	10	10	10	10	10	10	10	10	
		woman	66	64	66	66	66	66	66	66	
	3	child	30	30	30	30	30	30	30	30	
		woman	114	72	114	114	114	114	114	114	
male	1	child	3	3	3	3	3	3	3	3	
		man	119	98	119	119	119	119	119	119	
	2	child	9	9	9	9	9	9	9	9	
		man	99	90	99	99	99	99	99	99	
	3	child	28	28	28	28	28	28	28	28	
		man	319	225	319	319	319	319	319	319	

In [295...

```
df.groupby(['sex','pclass','embarked']).count()
```

Out[295...

sex	pclass	embarked	survived	age	sibsp	parch	fare	class	who	adult_male	decl
female	**1**	**C**	43	38	43	43	43	43	43	43	35
		Q	1	1	1	1	1	1	1	1	1
		S	48	44	48	48	48	48	48	48	43
	2	**C**	7	7	7	7	7	7	7	7	1
		Q	2	1	2	2	2	2	2	2	1
		S	67	66	67	67	67	67	67	67	8
	3	**C**	23	16	23	23	23	23	23	23	1
		Q	33	10	33	33	33	33	33	33	0
		S	88	76	88	88	88	88	88	88	5
male	**1**	**C**	42	36	42	42	42	42	42	42	31
		Q	1	1	1	1	1	1	1	1	1
		S	79	64	79	79	79	79	79	79	62
	2	**C**	10	8	10	10	10	10	10	10	1
		Q	1	1	1	1	1	1	1	1	0
		S	97	90	97	97	97	97	97	97	5
	3	**C**	43	25	43	43	43	43	43	43	0
		Q	39	14	39	39	39	39	39	39	1
		S	265	214	265	265	265	265	265	265	5

18. Select specific rows and columns

In [296...

```
df
```

Out[296...

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adu
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	
...	...	...	...	...	...	...	...	...	...	...	
886	0	2	male	27.0	0	0	13.0000	S	Second	man	
887	1	1	female	19.0	0	0	30.0000	S	First	woman	
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	
889	1	1	male	26.0	0	0	30.0000	C	First	man	
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	

891 rows × 15 columns

In [297...

```
df.head()
```

Out[297...

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

In [ ]:

```
#select columns
df[['sex','class']]
```

Out[ ]:

	sex	class
0	male	Third
1	female	First
2	female	Third
3	female	First
4	male	Third
...	...	...
886	male	Second
887	female	First
888	female	Third
889	male	First
890	male	Third

891 rows × 2 columns

In [299...

```
df[['sex','class','deck']]
```

Out[299...

	sex	class	deck
0	male	Third	NaN
1	female	First	C
2	female	Third	NaN
3	female	First	C
4	male	Third	NaN
...	...	...	...
886	male	Second	NaN
887	female	First	B
888	female	Third	NaN
889	male	First	C
890	male	Third	NaN

891 rows × 3 columns

In [300...

```
df.describe()
```

Out[300...

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

In [302...
```
df.describe().loc[['min','25%','50%', '75%','max']]
```

Out[302...

	survived	pclass	age	sibsp	parch	fare
min	0.0	1.0	0.420	0.0	0.0	0.0000
25%	0.0	2.0	20.125	0.0	0.0	7.9104
50%	0.0	3.0	28.000	0.0	0.0	14.4542
75%	1.0	3.0	38.000	1.0	0.0	31.0000
max	1.0	3.0	80.000	8.0	6.0	512.3292

In [305...
```
df.describe().loc['min':'max']
```

Out[305...

	survived	pclass	age	sibsp	parch	fare
min	0.0	1.0	0.420	0.0	0.0	0.0000
25%	0.0	2.0	20.125	0.0	0.0	7.9104
50%	0.0	3.0	28.000	0.0	0.0	14.4542
75%	1.0	3.0	38.000	1.0	0.0	31.0000
max	1.0	3.0	80.000	8.0	6.0	512.3292

In [306...
```
df.describe().loc['min':'max', :]
```

Out[306…

	survived	pclass	age	sibsp	parch	fare
min	0.0	1.0	0.420	0.0	0.0	0.0000
25%	0.0	2.0	20.125	0.0	0.0	7.9104
50%	0.0	3.0	28.000	0.0	0.0	14.4542
75%	1.0	3.0	38.000	1.0	0.0	31.0000
max	1.0	3.0	80.000	8.0	6.0	512.3292

In [307…
```
df.describe().loc['min':'max', 'survived']
```

Out[307…
```
min    0.0
25%    0.0
50%    0.0
75%    1.0
max    1.0
Name: survived, dtype: float64
```

In [308…
```
df.describe().loc['min':'max', 'survived':'age']
```

Out[308…

	survived	pclass	age
min	0.0	1.0	0.420
25%	0.0	2.0	20.125
50%	0.0	3.0	28.000
75%	1.0	3.0	38.000
max	1.0	3.0	80.000

19. Reshape multi-index series

In [309…
```
df.head()
```

Out[309…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

In [310…
```
df.survived.mean()
```

Out[310…
```
np.float64(0.3838383838383838)
```

In [311…
```
df.groupby('sex').survived.mean()
```

Out[311…
```
sex
female    0.742038
male      0.188908
Name: survived, dtype: float64
```

In [318…
```
df.groupby(['sex','class']).survived.mean()
```

```
C:\Users\Hp\AppData\Local\Temp\ipykernel_16784\1967038050.py:1: FutureWarning: The d
efault of observed=False is deprecated and will be changed to True in a future versi
on of pandas. Pass observed=False to retain current behavior or observed=True to ado
pt the future default and silence this warning.
  df.groupby(['sex','class']).survived.mean()
```

Out[318…
```
sex     class
female  First     0.968085
        Second    0.921053
        Third     0.500000
male    First     0.368852
        Second    0.157407
        Third     0.135447
Name: survived, dtype: float64
```

In [321…
```
df.groupby(["sex", "class"]).survived.mean().unstack()
```

```
C:\Users\Hp\AppData\Local\Temp\ipykernel_16784\2320923108.py:1: FutureWarning: The d
efault of observed=False is deprecated and will be changed to True in a future versi
on of pandas. Pass observed=False to retain current behavior or observed=True to ado
pt the future default and silence this warning.
  df.groupby(["sex", "class"]).survived.mean().unstack()
```

Out[321…

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

20. Convert continous to categorical data

In [322…
```
df.age.head()
```

Out[322…
```
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
Name: age, dtype: float64
```

In [327…
```
pd.cut(df.age, bins = [0, 18, 25, 99], labels = ['child','young_adult','adult']).he
df.head()
```

Out[327…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

In [328…

```
pd.cut(df.age, bins = [0, 18, 25, 99], labels = ['child','young_adult','adult']).he
df['new_age'] = pd.cut(df.age, bins = [0, 18, 25, 99], labels = ['child','young_adu
df.head()
```

Out[328…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

21. Convert one set of values into anothers

In [330…

```
df.sex.head()
```

Out[330…

```
0      male
1    female
2    female
3    female
4      male
Name: sex, dtype: object
```

In [333…

```
df.sex.map({'male':0,'female':1})
```

Out[333…

```
0      0
1      1
2      1
3      1
4      0
      ..
886    0
887    1
888    1
889    0
890    0
Name: sex, Length: 891, dtype: int64
```

In [334…
```
df['sex_num'] = df.sex.map({'male':0,'female':1})
df.head()
```

Out[334…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

In [335…
```
df.embarked.head()
```

Out[335…
```
0    S
1    C
2    S
3    S
4    S
Name: embarked, dtype: object
```

In [336…
```
df.embarked.count()
```

Out[336…
```
np.int64(889)
```

In [337…
```
df.embarked.unique()
```

Out[337…
```
array(['S', 'C', 'Q', nan], dtype=object)
```

In [338…
```
df.embarked.factorize()[0]
```

Out[338…

```
array([ 0,  1,  0,  0,  0,  2,  0,  0,  0,  1,  0,  0,  0,  0,  0,  0,  2,
        0,  0,  1,  0,  0,  2,  0,  0,  0,  1,  0,  2,  0,  1,  1,  2,  0,
        1,  0,  1,  0,  0,  1,  0,  0,  1,  1,  2,  0,  2,  2,  1,  0,  0,
        0,  1,  0,  1,  0,  0,  1,  0,  0,  1, -1,  0,  0,  1,  1,  0,  0,
        0,  0,  0,  0,  0,  1,  0,  0,  0,  0,  0,  0,  0,  0,  2,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,  1,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  2,  0,  1,  0,  0,  1,  0,  2,  0,  1,
        0,  0,  0,  1,  0,  0,  1,  2,  0,  1,  0,  1,  0,  0,  0,  0,  1,
        0,  0,  0,  1,  1,  0,  0,  2,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  1,  2,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  2,  0,  0,  1,  0,  0,  1,  0,  0,  0,  1,  0,  0,  0,  0,  2,
        0,  2,  0,  0,  0,  0,  0,  1,  1,  2,  0,  2,  0,  0,  0,  0,  1,
        0,  0,  0,  1,  2,  1,  0,  0,  0,  0,  2,  1,  0,  0,  1,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  1,  2,  0,  0,  1,  2,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        1,  1,  0,  1,  0,  2,  0,  0,  0,  2,  0,  0,  0,  0,  0,  0,  0,
        0,  1,  2,  0,  0,  0,  2,  0,  2,  0,  0,  0,  0,  1,  0,  0,  0,
        2,  0,  1,  1,  0,  0,  1,  1,  0,  0,  1,  2,  2,  0,  2,  0,  0,
        1,  1,  1,  1,  1,  1,  0,  0,  0,  0,  0,  0,  0,  1,  0,  0,  2,
        0,  0,  1,  0,  0,  0,  1,  2,  0,  0,  0,  0,  0,  0,  1,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,  0,  1,  0,  0,
        0,  2,  2,  0,  1,  1,  0,  2,  0,  1,  1,  2,  1,  1,  0,  0,  1,
        0,  1,  0,  1,  1,  0,  1,  1,  0,  0,  0,  0,  0,  0,  2,  1,  0,
        0,  0,  1,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  2,  2,  0,  0,  0,  0,  0,  0,  0,  1,  2,  0,  0,  0,
        0,  0,  0,  2,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  0,  0,  0,  0,  0,  1,  0,  0,  0,  1,  1,  0,  1,  0,  0,  0,
        2,  0,  0,  0,  0,  0,  0,  0,  0,  2,  1,  0,  0,  0,  1,  0,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  1,  0,  0,  1,  0,  0,  0,  0,  0,
        1,  0,  1,  1,  0,  0,  0,  0,  2,  2,  0,  0,  1,  0,  0,  0,  0,
        2,  0,  0,  1,  0,  0,  0,  2,  0,  0,  0,  0,  1,  1,  1,  2,  0,
        0,  0,  0,  0,  1,  1,  1,  0,  0,  0,  1,  0,  1,  0,  0,  0,  0,
        1,  0,  0,  1,  0,  0,  1,  0,  2,  1,  0,  0,  1,  1,  0,  0,  2,
        0,  0,  0,  0,  0,  0,  0,  1,  0,  0,  0,  0,  2,  0,  0,  0,  0,
        1,  0,  0,  1,  0,  1,  1,  0,  0,  1,  0,  0,  0,  1,  0,  2,  0,
        0,  0,  0,  1,  1,  0,  0,  0,  0,  1,  0,  0,  0,  1,  0,  0,  0,
        2,  2,  0,  0,  0,  0,  0,  0,  1,  0,  1,  0,  0,  0,  2,  0,  0,
        2,  0,  0,  1,  0,  0,  0,  0,  0,  0,  0,  0,  1,  0,  0,  1,  1,
        0,  1,  0,  0,  0,  0,  0,  2,  2,  0,  0,  2,  0,  1,  0,  1,  0,
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,
        2,  1,  0,  0,  0,  1,  0,  0,  0,  0,  0,  1,  0,  1,  0,  0,  0,
        2,  1,  0,  1,  0,  1,  2,  0,  0,  0,  0,  0,  1,  1,  0,  0,  0,
        0,  0,  1,  0,  2,  0,  0,  0,  0,  0,  0,  0,  0,  2,  0,  0,  0,
        1,  0,  0,  0,  0,  0,  1,  0,  0,  0,  0,  1,  0,  0,  0,  0,  0,
        0,  2,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,  0,  0,
        0,  1,  2,  2,  0,  0,  0,  0,  1,  0,  0,  2,  0,  2,  0,  1,  0,
        0,  0,  0,  0,  0,  2,  0,  1,  2,  0,  0,  1,  0,  0,  0,  0,  1,
        0,  0,  0,  0,  1,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
        0,  1,  0,  0,  0,  0,  0,  0,  0,  2,  0,  1,  2, -1,  1,  0,  1,
        0,  0,  1,  0,  0,  0,  1,  0,  0,  1,  1,  0,  0,  0,  1,  0,  1,
        0,  0,  1,  0,  0,  0,  0,  0,  1,  1,  0,  0,  0,  0,  0,  0,  1,
        0,  0,  0,  0,  0,  0,  0,  1,  1,  0,  0,  0,  1,  0,  0,  0,  0,
        0,  2,  0,  0,  0,  1,  2])
```

In [339…

```
df['embarked_number'] = df.embarked.factorize()[0]
```

```
df.head()
```

Out[339…

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_m
0	0	3	male	22.0	1	0	7.2500	S	Third	man	T
1	1	1	female	38.0	1	0	71.2833	C	First	woman	Fa
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	Fa
3	1	1	female	35.0	1	0	53.1000	S	First	woman	Fa
4	0	3	male	35.0	0	0	8.0500	S	Third	man	T

22. Transpose a whide dataframe

In [345…

```
import numpy as np
import pandas as pd
```

In [356…

```
df = pd.DataFrame(np.random.rand(200,25), columns=list('abcdefghijklmnopqrstuvwxy'))
df.head()
```

Out[356…

	a	b	c	d	e	f	g	h	i
0	0.714746	0.880579	0.934230	0.747476	0.250086	0.016927	0.637038	0.746211	0.847617
1	0.206297	0.929607	0.607055	0.031078	0.170622	0.376513	0.474965	0.925821	0.063452
2	0.329756	0.277162	0.785262	0.395048	0.654646	0.852857	0.871783	0.987413	0.598360
3	0.049474	0.276890	0.542986	0.301527	0.435884	0.226238	0.703224	0.730651	0.734326
4	0.919218	0.328134	0.724551	0.451633	0.345794	0.590800	0.332888	0.310333	0.701419

5 rows × 25 columns

In [357…

```
df.head(10).T
```

Out[357…

	0	1	2	3	4	5	6	7	8
a	0.714746	0.206297	0.329756	0.049474	0.919218	0.708708	0.528903	0.111598	0.931608
b	0.880579	0.929607	0.277162	0.276890	0.328134	0.200812	0.612130	0.400527	0.583915
c	0.934230	0.607055	0.785262	0.542986	0.724551	0.272404	0.015854	0.518318	0.185198
d	0.747476	0.031078	0.395048	0.301527	0.451633	0.354443	0.769722	0.129250	0.228593
e	0.250086	0.170622	0.654646	0.435884	0.345794	0.368933	0.050054	0.162961	0.067384
f	0.016927	0.376513	0.852857	0.226238	0.590800	0.789674	0.683336	0.254430	0.014301
g	0.637038	0.474965	0.871783	0.703224	0.332888	0.142863	0.593075	0.701797	0.067542
h	0.746211	0.925821	0.987413	0.730651	0.310333	0.290988	0.503405	0.492329	0.093039
i	0.847617	0.063452	0.598360	0.734326	0.701419	0.036489	0.814056	0.913128	0.432973
j	0.675809	0.890455	0.229260	0.356421	0.667807	0.807176	0.629221	0.587452	0.754876
k	0.233720	0.170707	0.708667	0.314061	0.946020	0.792601	0.054144	0.586330	0.513430
l	0.891530	0.049928	0.296700	0.704552	0.996536	0.578093	0.370853	0.381413	0.496348
m	0.624854	0.212529	0.818649	0.225854	0.625074	0.658790	0.627916	0.697669	0.734129
n	0.181665	0.719016	0.269218	0.072096	0.748253	0.636784	0.792440	0.884775	0.743773
o	0.833598	0.925150	0.042374	0.265999	0.719622	0.001297	0.828582	0.926515	0.898028
p	0.537626	0.263173	0.540599	0.253337	0.151183	0.537317	0.992437	0.068684	0.318521
q	0.797838	0.836849	0.221699	0.367817	0.865206	0.021029	0.603355	0.693663	0.456371
r	0.221447	0.558674	0.188263	0.798955	0.441953	0.580764	0.293882	0.701808	0.275532
s	0.975357	0.194890	0.407122	0.969411	0.620383	0.363349	0.838249	0.996651	0.617022
t	0.466088	0.884739	0.439873	0.243583	0.032834	0.543304	0.366936	0.562778	0.036829
u	0.654660	0.923759	0.407698	0.908603	0.518084	0.389271	0.708782	0.989818	0.251023
v	0.788391	0.107066	0.287135	0.803043	0.527640	0.015113	0.112050	0.321722	0.115319
w	0.357217	0.701124	0.538539	0.919095	0.963398	0.809344	0.165713	0.980514	0.380860
x	0.864920	0.177876	0.748884	0.905261	0.965300	0.667590	0.811416	0.739965	0.868750
y	0.696008	0.344803	0.247522	0.035524	0.064684	0.804748	0.140961	0.450174	0.670337

In [358…

```
df.describe()
```

Out[358…

	a	b	c	d	e	f	g
count	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000
mean	0.532849	0.462055	0.484121	0.509875	0.453350	0.482631	0.464995
std	0.292598	0.273876	0.293190	0.276846	0.302220	0.279336	0.310441
min	0.003548	0.000501	0.002814	0.021809	0.001545	0.002977	0.001102
25%	0.285684	0.233281	0.233062	0.264593	0.166075	0.248652	0.175778
50%	0.534126	0.448426	0.487098	0.506454	0.437920	0.479605	0.437777
75%	0.773307	0.681988	0.751641	0.745343	0.723682	0.721787	0.741897
max	0.998871	0.993437	0.987450	0.975403	0.998330	0.995693	0.998168

8 rows × 25 columns

In [359…

```
df.describe().T
```

Out[359…

	count	mean	std	min	25%	50%	75%	max
a	200.0	0.532849	0.292598	0.003548	0.285684	0.534126	0.773307	0.998871
b	200.0	0.462055	0.273876	0.000501	0.233281	0.448426	0.681988	0.993437
c	200.0	0.484121	0.293190	0.002814	0.233062	0.487098	0.751641	0.987450
d	200.0	0.509875	0.276846	0.021809	0.264593	0.506454	0.745343	0.975403
e	200.0	0.453350	0.302220	0.001545	0.166075	0.437920	0.723682	0.998330
f	200.0	0.482631	0.279336	0.002977	0.248652	0.479605	0.721787	0.995693
g	200.0	0.464995	0.310441	0.001102	0.175778	0.437777	0.741897	0.998168
h	200.0	0.514448	0.278119	0.003678	0.260404	0.545420	0.735296	0.997068
i	200.0	0.493428	0.279403	0.010259	0.271080	0.491105	0.739318	0.987621
j	200.0	0.496081	0.287544	0.001869	0.248141	0.497732	0.727809	0.980752
k	200.0	0.509130	0.304209	0.002926	0.229706	0.527946	0.783611	0.988370
l	200.0	0.494507	0.260378	0.030513	0.285903	0.489514	0.685495	0.998939
m	200.0	0.523489	0.280010	0.000377	0.295979	0.508762	0.743113	0.993624
n	200.0	0.513824	0.284486	0.001311	0.255350	0.553286	0.741962	0.997327
o	200.0	0.520173	0.286145	0.001297	0.268578	0.544845	0.768386	0.990380
p	200.0	0.501075	0.288890	0.001836	0.271755	0.480958	0.779745	0.992437
q	200.0	0.468916	0.288464	0.001079	0.233099	0.444957	0.698418	0.998851
r	200.0	0.485921	0.292976	0.000384	0.240300	0.482890	0.741227	0.997255
s	200.0	0.504873	0.275556	0.000541	0.283866	0.493993	0.722730	0.997851
t	200.0	0.495539	0.278326	0.002431	0.261162	0.508466	0.741961	0.993099
u	200.0	0.526382	0.290915	0.003058	0.305396	0.529500	0.755489	0.994876
v	200.0	0.481333	0.297196	0.000055	0.205359	0.459240	0.743431	0.993321
w	200.0	0.500580	0.285008	0.001315	0.252571	0.513521	0.735609	0.999619
x	200.0	0.507976	0.293997	0.007876	0.239606	0.537152	0.755405	0.989984
y	200.0	0.492803	0.289580	0.005227	0.254346	0.482496	0.733664	0.997767

In [360…

```
# save transposed data
df.describe().T.to_csv('decribe.csv')
```

23. Reshaping a dataframe

In [409…

```
fasla = pd.DataFrame([['2345', 100, 200, 300], ['34567', 400, 500, 600],['67890', 7
                      columns=['zip', 'factory', 'warehouse', 'retail']
```

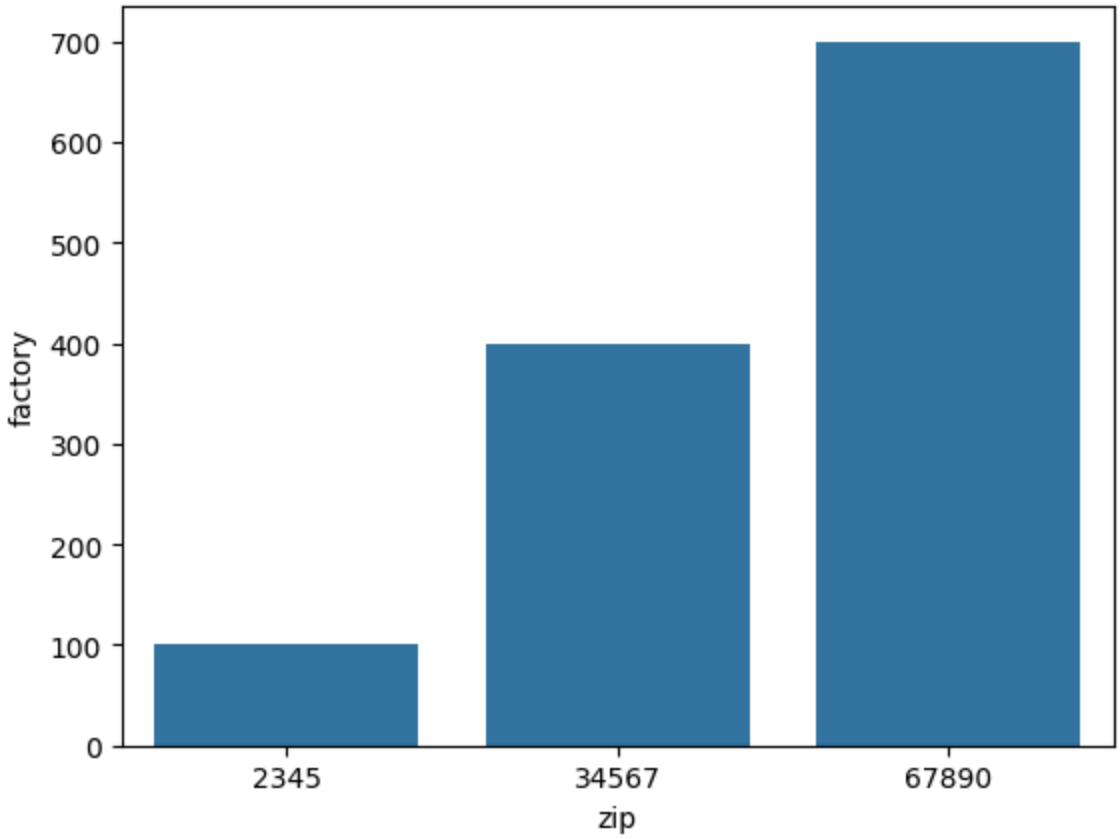
```
)

fasla.head()
```

Out[409…

	zip	factory	warehouse	retail
0	2345	100	200	300
1	34567	400	500	600
2	67890	700	800	900

In [421…
```
sns.barplot(x="zip", y="factory",data=fasla)
```

Out[421…
```
<Axes: xlabel='zip', ylabel='factory'>
```



In [410…
```
fasla.head().T
```

Out[410…

	0	1	2
zip	2345	34567	67890
factory	100	400	700
warehouse	200	500	800
retail	300	600	900

In [411…

```
# fasla2 = pd.DataFrame([[1,'12345', 'factory'], [2, '34567', 'warehouse']],
#     columns=['user_id', 'zip', 'location_type'])

# fasla2.head()
```

In [412…

```
fasla.melt(id_vars='zip',var_name='locatio_type', value_name='distance')
```

Out[412…

	zip	locatio_type	distance
0	2345	factory	100
1	34567	factory	400
2	67890	factory	700
3	2345	warehouse	200
4	34567	warehouse	500
5	67890	warehouse	800
6	2345	retail	300
7	34567	retail	600
8	67890	retail	900

In [426…

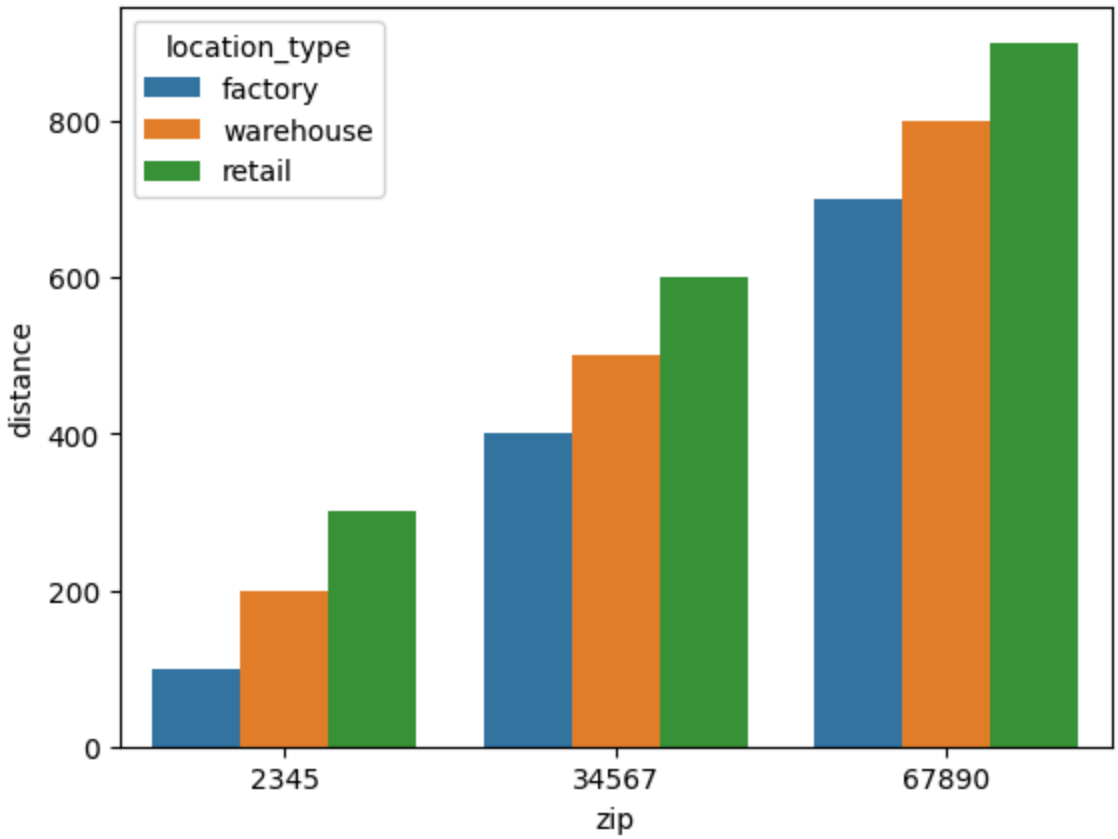
```
fasla_long = fasla.melt(
    id_vars="zip",
    var_name="location_type",
    value_name="distance"
)
```

In [425…

```
sns.barplot(
    data=fasla_long,
    x="zip",
    y="distance",
    hue="location_type"
)
```

Out[425…

```
<Axes: xlabel='zip', ylabel='distance'>
```



In [414…

```
fasla_long.dtypes
```

Out[414…

```
zip             object
locatio_type    object
distance         int64
dtype: object
```